```
  }
}
```

❶ Stereotype annotation marking a class as a Spring component and a source of handlers for REST endpoints

❷ Identifies the root path mapping URI for which the class provides handlers ("/books")

❸ Maps HTTP GET requests to the specific handler method

❹ A URI template variable appended to the root path mapping URI ("/books/{isbn}")

❺ @PathVariable binds a method parameter to a URI template variable ({isbn}).

❻ Maps HTTP POST requests to the specific handler method

❼ Returns a 201 status if the book is created successfully

❽ @RequestBody binds a method parameter to the body of a web request.

❾ Maps HTTP DELETE requests to the specific handler method

❿ Returns a 204 status if the book is deleted successfully

Maps HTTP PUT requests to the specific handler method

Go ahead and run the application (`./gradlew bootRun`). When verifying HTTP interactions with an application, you can either use a command-line tool like `curl` or software with a graphical user interface like Insomnia. I'm going to use a convenient command-line tool called HTTPie (https://httpie.org). You can find information about how to install it in section A.4 of appendix A.

Open a Terminal window and perform an HTTP `POST` request to add a book to the catalog:

```
$ http POST :9001/books author="Lyra Silverstar" \
    title="Northern Lights" isbn="1234567891" price=9.90
```

The result should be an HTTP response with the `201` code, meaning that the book has been created successfully. Let's double-check by submitting an HTTP `GET` request to fetch the book with the ISBN code we used on creation.